# BATCH — подсистема пакетного исполнения

Руководство пользователя

BATCH-UG-0102-RUS

BATCH Version 1.0.1 *** 29-JUL-2026

# Содержание

**Предисловие** **3**
Кому адресовано . . . 3
Состав руководства . . . 4
Обозначения . . . 5

**Глава 1. Введение** **6**
1.1 Назначение . . . 6
1.2 Составные части . . . 7
1.3 Отличия от повременного планировщика . . . 8
1.4 Состояния задания . . . 9
1.5 Жизненный цикл задания . . . 10

**Глава 2. Установка** **11**
2.1 Требования . . . 11
2.2 Сборка и установка . . . 12
2.3 Что устанавливается . . . 13
2.4 Библиотека помощи . . . 14
2.5 Запуск контроллера вместе с системой . . . 15
2.6 Проверка установки . . . 16
2.7 Удаление . . . 17

**Глава 3. Настройка** **18**
3.1 Ключи контроллера заданий . . . 18
3.2 Изменение службы . . . 19
3.3 Настройки, задаваемые при конфигурации . . . 20
3.4 Где ищет утилита . . . 21
3.5 Что задание наследует . . . 22

**Глава 4. Работа с заданиями** **23**
4.1 Подача задания . . . 23
4.2 Передача данных заданию . . . 24
4.3 Вывод задания . . . 25
4.4 Управление временем запуска . . . 26
4.5 Просмотр очереди . . . 28
4.6 Снятие задания . . . 29

**Глава 5. Словарь команд** **30**
BATCH SUBMIT . . . 31
BATCH SHOW . . . 34
BATCH DELETE . . . 36
BATCH RELEASE . . . 37
BATCH SET ENTRY . . . 38
BATCH PURGE . . . 40
BATCH SYNC . . . 41
BATCH QUEUE . . . 42
BATCH ANALYZE . . . 43

BATCHD . . . . . 44

**Глава 6. Сообщения** **46**
6.1 Раскладка значения условия . . . . . 46
6.2 Вид сообщения . . . . . 47
6.3 Справочник сообщений . . . . . 48

**Глава 7. Устранение неполадок** **56**
7.1 Контроллер недоступен . . . . . 57
7.2 Контроллер не запускается . . . . . 58
7.3 Задание не начинается . . . . . 59
7.4 Задание завершилось неудачей . . . . . 60
7.5 После перезапуска очередь пуста . . . . . 61
7.6 Каталог журналов растёт . . . . . 62
7.7 Команда отвергнута . . . . . 63
7.8 Контроллер медленно отвечает . . . . . 64
7.9 Что собрать для сообщения об ошибке . . . . . 65

**Приложение A. Файлы** **66**
Переменные окружения . . . . . 67

**Приложение B. Формы спецификации времени** **68**
B.1 Разностное время . . . . . 69
B.2 Абсолютное время . . . . . 70
B.3 Ключевые слова . . . . . 71

# Предисловие

## Кому адресовано

Руководство предназначено тому, кто подаёт сценарии оболочки на отложенное исполнение, и тому, кто устанавливает подсистему и следит за её работой. Предполагается знакомство с оболочкой POSIX и ничего более. Знание OpenVMS не требуется, хотя читатель, знакомый с командами `SUBMIT` и `SHOW QUEUE`, узнает и терминологию, и вид командной строки.

Программисту, которому предстоит вносить изменения, адресовано *Руководство программиста*. Здесь о внутреннем устройстве не говорится.

## Состав руководства

| Часть | Содержание |
|---|---|
| Глава 1 | Введение: назначение подсистемы, её составные части, жизненный цикл задания |
| Глава 2 | Установка подсистемы и запуск контроллера вместе с системой |
| Глава 3 | Настройка контроллера и то, что задание от него наследует |
| Глава 4 | Как выполняются обычные действия с заданиями |
| Глава 5 | Словарь команд, по описанию на каждую |
| Глава 6 | Вид сообщения и справочник: все сообщения с пояснением и требуемым действием |
| Глава 7 | Устранение неполадок: признак, куда смотреть, что делать |
| Приложение A | Файлы, создаваемые и используемые во время работы |
| Приложение B | Допустимые формы спецификации времени |

Тому, кто ставит подсистему впервые, нужны Глава 2, затем Глава 3. Тому, у кого она работает и кто хочет ею пользоваться, — Глава 4. Тому, у кого на руках сообщение, — Глава 6, а у кого признак неисправности, — Глава 7.

# Обозначения

| Обозначение | Смысл |
|---|---|
| `$` | Приглашение оболочки. Далее следует то, что вводит пользователь. |
| *курсив* | Определяемый термин либо величина, подлежащая замене. |
| ПРОПИСНЫЕ | Команда, квалификатор или мнемоника условия. |
| `[ ]` | Необязательная часть записи. Скобки не набираются. |
| `{ }` | Обязательный выбор из перечисленного. Скобки не набираются. |
| `, . . .` | Предшествующий элемент может повторяться. |

Команды и квалификаторы приведены прописными для удобочитаемости, но набирать их можно в любом регистре.

**Замечание**

Примеры вывода приведены в точности так, как их выдаёт программа: её сообщения на английском языке, и перевод в них внёс бы расхождение с тем, что читатель увидит на экране.

# Глава 1. Введение

## 1.1 Назначение

Подсистема BATCH выполняет сценарии оболочки как *пакетные задания*. Пакетное задание — это сценарий, помещённый в очередь и выполняемый позднее, без терминала, под управлением постоянно работающего процесса, который называется *контроллером заданий*.

Подача сценария не есть его выполнение. При подаче в очередь заводится *запись*, и её номер возвращается подавшему; этот номер опознаёт задание до конца его существования. Когда задание действительно начнёт выполняться, решает контроллер — исходя из числа уже выполняющихся заданий, из приоритета задания и из времени, раньше которого его запускать не велено.

## 1.2 Составные части

Подсистема состоит из двух образов:

| Образ | Назначение |
|---|---|
| `batch` | Утилита. Разбирает команды, передаёт запросы контроллеру, показывает результат. |
| `batchd` | Контроллер заданий. Владеет очередью, планирует задания, порождает процессы, отмечает их завершение. |

Образы общаются через сокет домена UNIX в корневом каталоге подсистемы. Контроллер устанавливает личность отправителя каждого запроса по данным, которые сообщает ядро, и не доверяет тому, что клиент говорит о себе сам.

## 1.3 Отличия от повременного планировщика

Читателю, знакомому с `cron`, стоит отметить следующее:

- Задание опознаётся номером записи, назначенным при подаче, а не идентификатором процесса. Номер записи существует раньше, чем какой-либо процесс.
- Очередь имеет *состояния*. Задание может ожидать свободного места для исполнения либо назначенного времени начала.
- Число одновременно выполняющихся заданий ограничено. Подача большего числа заданий, чем допускает предел, систему не перегружает: лишние ждут.
- Задание, завершившееся ненормально, **остаётся в очереди** вместе со своим журналом и показывается командой `SHOW` без всяких квалификаторов.

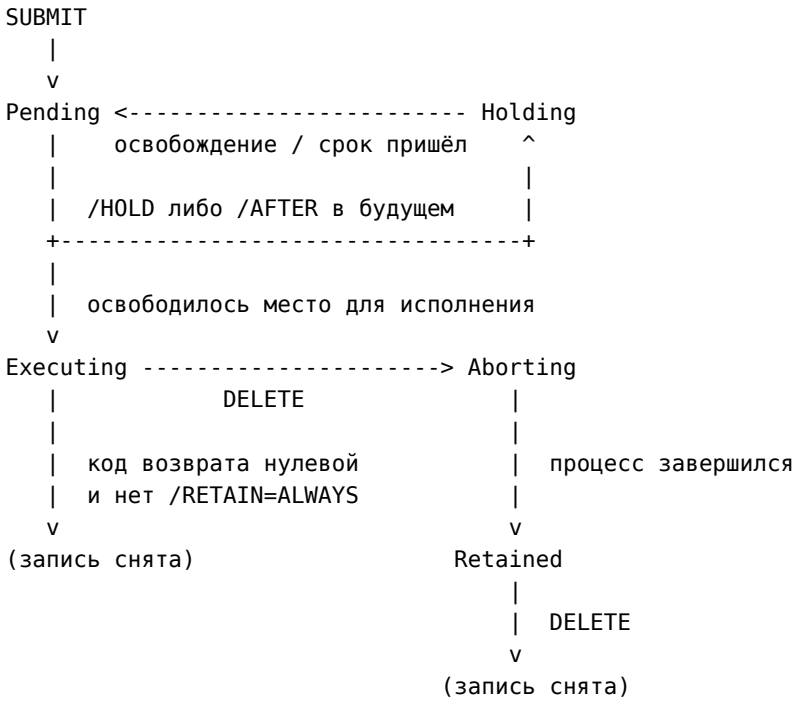
## 1.4 Состояния задания

Задание находится ровно в одном из следующих состояний:

| Состояние | Смысл |
|---|---|
| `Pending` | Задание может быть запущено и ожидает свободного места для исполнения. |
| `Holding` | Задание удержано — квалификатором `/HOLD` либо потому, что время `/AFTER` ещё не наступило. |
| `Executing` | Для задания порождён процесс, и он работает. |
| `Aborting` | Заданию послан сигнал снятия, и оно ещё не завершилось. |
| `Retained` | Задание завершилось, а его запись сохранена для просмотра. |

## 1.5 Жизненный цикл задания

Допустимые переходы таковы:




Задание, завершившееся успешно, очередь покидает — так же, как в OpenVMS. Чтобы запись успешного задания сохранилась, подавайте его с `/RETAIN=ALWAYS`.

# Глава 2. Установка

## 2.1 Требования

| | |
|---|---|
| Операционная система | Linux. Подсистема опирается на сокеты домена UNIX, параметр сокета `SO_PEERCRED`, `daemon(3)` и `poll(2)`; сборка вне Linux прекращается на этапе конфигурации. |
| Средства сборки | CMake 3.16 или новее и транслятор Си со поддержкой gnu11. |
| Супервизор служб | systemd, если контроллер должен запускаться вместе с системой. В остальном не требуется. |
| Права | Установка пишет в системные каталоги и требует привилегированной учётной записи. Сборка и прогон тестов — нет. |

Заранее должны быть установлены два пакета: **StarLet**, то есть UTILITY$ROUTINES, и **HELP**. Ни один из них не поставляется с этими исходными текстами и ни один не является необязательным: без первого не собирается ничего, без второго — команда `BATCH HELP`.

## 2.2 Сборка и установка

```
$ cmake -S . -B build
$ cmake --build build -j
$ ctest --test-dir build --output-on-failure
# cmake --install build
```

Тесты стоит прогнать до установки. Они поднимают собственный контроллер на временном каталоге, проверяют работу очереди и снимают его; ни рабочей конфигурации, ни чьей-либо учётной записи они не касаются.

Чтобы поставить не в `/usr/local`, укажите это при конфигурации:

```
$ cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/batch
```

Прочие настройки, задаваемые при конфигурации, перечислены в разделе 3.3.

## 2.3 Что устанавливается

| Путь | Содержимое |
|---|---|
| *префикс*`/bin/batch` | утилита |
| *префикс*`/sbin/batchd` | контроллер заданий |
| *префикс*`/share/doc/BATCH/` | руководства и краткое описание, в markdown и, если запрошено, в PDF |
| `/lib/systemd/system/batchd.service` | unit-файл контроллера |
| `/var/spool/batch/` | корневой каталог подсистемы: журнал очереди и сокет |
| `/var/spool/batch/log/` | журналы заданий |

Место unit-файла задаётся `BATCH_K_SYSTEMDDIR`; `BATCH_M_SYSTEMD=OFF` исключает его вовсе, что и нужно системе без systemd.

## 2.4 Библиотека помощи

Установка кладёт библиотеку помощи подсистемы в `share/help` и перестраивает каталог, который утилита HELP разделяет между своими пользователями:

```
-- %BATCH-I-SYSCACHE, the help catalogue has been rebuilt
```

Этот каталог собирается из библиотек каталога однажды, поэтому библиотека, установленная после того, как он был построен, не видна, пока он не построен заново. Отсюда два следствия.

**Промежуточная установка** — сделанная с заданной `DESTDIR`, при сборке пакета, — каталога не строит и сообщает об этом: каталог, построенный из промежуточного дерева, описывал бы его, а не систему. Перестраивает его тот, кто ставит пакет, обыкновенно из послеустановочного сценария:

> Образ просмотрщика HELP называется `vhelp`, а его библиотекарь — `vhelptran`: `help` есть встроенная команда оболочки bash и ей подобных, а встроенную команду оболочка ищет прежде пути, так что до образа с таким именем не добраться никаким порядком в `PATH`. Подсистема HELP поставляет сценарий, отключающий встроенную команду для тех, кто предпочитает набирать `help`.

```
# vhelp /SYSCACHE
```

**Неудача перестроения** сообщается и обходится стороной. Стоит она лишь одного: `help submit`, спрошенный у общей утилиты, не находит темы, пока каталог не построен. Команды `BATCH HELP` это не касается вовсе — она читает библиотеку напрямую и никакого каталога не спрашивает.

## 2.5 Запуск контроллера вместе с системой

Unit-файл устанавливается, но **не включается**. Запускать ли контроллер вместе с системой — дело того, кто ею управляет.

```
# systemctl enable --now batchd
# systemctl status batchd
```

Под супервизором контроллер работает на переднем плане, поэтому его диагностика попадает в журнал системы, а не в отдельный файл:

```
# journalctl -u batchd -f
```

**Замечание**

> В unit-файле указано `KillMode=process`, поэтому остановка службы посылает сигнал только контроллеру. Задание живёт в собственном сеансе и должно переживать контроллер; принятое по умолчанию снятие всей управляющей группы унесло бы с собой каждое выполняющееся задание.

Без systemd контроллер запускается как всякий иной демон и отцепляется сам:

```
# /usr/local/sbin/batchd /ROOT=/var/spool/batch /JOB_LIMIT=8
```

## 2.6 Проверка установки

```
$ printf 'echo it works\n' > /tmp/probe.sh
$ batch submit /tmp/probe.sh /NAME=probe /KEEP
Job probe (queue SYS$BATCH, entry 1) queued

$ batch show /all
There are no jobs in the queue

$ cat /var/spool/batch/log/probe_1.log
it works
```

Очередь пуста потому, что задание завершилось успешно и его запись очередь покинула, — так и описано в разделе 1.5. Журнал сохранился, поскольку был указан `/KEEP`.

Если первая команда ответила `%BATCH-E-NOCONNECT`, контроллер не запущен; об этом раздел 7.1.

## 2.7 Удаление

```
# cmake --build build --target uninstall
```

Что было установлено, читается из перечня, записанного при установке, — ничего не угадывается и не трогается лишнего. Корневой каталог подсистемы и каталог журналов сносятся **только пустыми**: удаление программы не повод выбрасывать журнал очереди и журналы заданий.

Отключение службы оставлено администратору и упоминается в конце удаления:

```
# systemctl disable --now batchd
```

# Глава 3. Настройка

## 3.1 Ключи контроллера заданий

Контроллер получает настройки из своей командной строки. Где их указывать, когда он работает службой, сказано в разделе 3.2.

| Ключ | По умолчанию | Смысл |
|---|---|---|
| `/ROOT=`*`каталог`* | `/var/spool/batch` | Корневой каталог подсистемы. Там лежат журнал очереди, сокет и каталог журналов. |
| `/JOB_LIMIT=`*`n`* | 16 | Сколько заданий может выполняться одновременно. От числа процессоров не зависит. |
| `/LCORE=`*`список`* | — | Ограничивает пакетную нагрузку названными процессорами и раскладывает её по ним, по одному на задание в свой черёд. Принимаются обе записи: `(0,3,4)` и `0,3,4`. Задание со своим `/CPU` сохраняет названное. |
| `/QUEUE=`*`имя`* | `SYS$BATCH` | Имя очереди по умолчанию. |
| `/LOG_RETAIN=`*`дней`* | 7 | Сколько хранится оставшийся журнал задания. Ноль отключает уборку. |
| `/FOREGROUND` | — | Не отцепляться. Для супервизора служб и для проверок. |
| `/VERBOSE` | — | Выдавать и сообщения информационной серьёзности. |

Ключ вводится косой чертой либо дефисом и сокращается до любого однозначного начала; первые буквы у всех разные, поэтому годятся `-r=`, `-j=`, `-q=`, `-l=`, `-f` и `-v`. Значение присоединяется к ключу знаком равенства и никогда не отделяется от него пробелом.

## 3.2 Изменение службы

Установленный unit-файл править не следует: следующая установка его перезапишет. Пользуйтесь накладкой.

```
# systemctl edit batchd
```

```
[Service]
ExecStart=
ExecStart=/usr/local/sbin/batchd /FOREGROUND /ROOT=/var/spool/batch /JOB_LIMIT=8 /LOG_RETAIN=30
```

Пустая строка `ExecStart=` обязательна: без неё вторая строка добавит команду, а не заменит ту, которая в unit-файле.

```
# systemctl daemon-reload
# systemctl restart batchd
```

Перезапуск контроллера очередь не теряет — она читается обратно из журнала, — но задание, которое выполнялось, окажется осиротевшим, если оно не было подано с `/RESTART`. Об этом говорится в описании команды SUBMIT в Главе 5.

## 3.3 Настройки, задаваемые при конфигурации

Эти значения вкомпилированы и меняются повторной конфигурацией сборки:

| Переменная | По умолчанию | Смысл |
|---|---|---|
| `BATCH_K_SPOOLROOT` | `/var/spool/batch` | Корневой каталог, который образы принимают, если он не назван |
| `BATCH_K_SHELL` | `/bin/sh` | Интерпретатор команд, порождаемый для задания |
| `BATCH_K_FACILITY` | `BATCH` | Имя подсистемы, появляющееся в сообщениях |
| `BATCH_K_SYSTEMDDIR` | `/lib/systemd/system` | Куда ставится unit-файл |
| `BATCH_M_SYSTEMD` | `ON` | Ставить ли unit-файл вообще |
| `BATCH_M_PDF` | `OFF` | Верстать ли руководства в PDF и ставить ли их |
| `BATCH_M_TESTS` | `ON` | Собирать ли тесты |
| `BATCH_M_STRICT` | `OFF` | Считать ли предупреждения транслятора ошибками |

```
$ cmake -S . -B build -DBATCH\$K_SPOOLROOT=/opt/batch -DBATCH\$M_SYSTEMD=OFF
```

Обратная косая черта нужна лишь затем, чтобы оболочка не раскрыла знак доллара; сам CMake допускает его в имени.

## 3.4 Где ищет утилита

Утилита берёт корневой каталог подсистемы из переменной окружения `BATCH_ROOT`, а если та не установлена — из вкомпилированного значения. Поэтому контроллеру, запущенному с `/ROOT`, отличным от принятого по умолчанию, требуется, чтобы его пользователи установили переменную соответственно:

```
$ export BATCH_ROOT=/opt/batch
```

Расхождение проявляется сообщением `%BATCH-E-NOCONNECT`: утилита ищет сокет в каталоге, где его не создавали.

## 3.5 Что задание наследует

Задание порождается контроллером и берёт от него то, чего не называет само:

| | |
|---|---|
| Привязка к процессорам | Наследуется. Ключ `/CPU` задаёт её для отдельного задания. |
| Значение nice | Наследуется. Для отдельного задания не задаётся. |
| Ограничения ресурсов | Наследуются. Для отдельного задания не задаются. |
| Управляющая группа | Наследуется. Для отдельного задания не задаётся. |
| Личность | **Не** наследуется: задание работает от имени того, кто его подал. |
| Рабочий каталог | **Не** наследуется: задание начинается там, откуда было подано. |
| Окружение | **Не** наследуется: заданию даются `BATCH_*`, `P1` … `P8` и ничего более из окружения контроллера. |

Маску привязки, под которой работает сам контроллер, он сообщает при запуске, чтобы контроллер, запущенный под `taskset` или с `CPUAffinity=` в unit-файле, не стеснял свои задания незаметно:

```
%BATCH-I-INFO, the controller runs on processors 0-3, and a job inherits that
```

Заметьте, что ограничение управляющей группы, например `CPUQuota=`, действует на контроллер и все порождённые им задания в совокупности.

# Глава 4. Работа с заданиями

## 4.1 Подача задания

Для подачи сценария применяется команда `SUBMIT`:

```
$ batch submit /opt/jobs/nightly.sh
Job nightly (queue SYS$BATCH, entry 17) queued
```

Команда сообщает назначенное заданию имя, очередь, в которую оно помещено, и номер записи. Номер стоит запомнить: он требуется командам `DELETE` и `RELEASE`.

По умолчанию именем задания становится имя файла сценария без окончания `.sh`. Иное имя задаётся квалификатором `/NAME`:

```
$ batch submit /opt/jobs/nightly.sh /NAME=reindex
Job reindex (queue SYS$BATCH, entry 18) queued
```

Спецификация сценария разворачивается в абсолютный путь при вводе команды. Заметьте также, что задание начинается в **том рабочем каталоге, который был текущим при подаче**, а не в том, который будет текущим к началу исполнения. Поэтому сценарий, опирающийся на относительные пути, поведёт себя так же, как при запуске вручную из того же каталога.

## 4.2 Передача данных заданию

Квалификатор `/PARAMETERS` передаёт заданию до восьми значений:

```
$ batch submit /opt/jobs/backup.sh '/PARAMETERS=(full,/srv/data,verbose)'
```

Каждое значение доступно сценарию двумя путями: как переменные окружения `P1` … `P8` и как позиционные доводы `$1` … `$8`.

```
#!/bin/sh
echo "mode  = ${P1}"          # full
echo "target= $2"             # /srv/data
```

Значения передаются интерпретатору отдельными элементами вектора доводов. Они никогда не сливаются в командную строку, поэтому значение, содержащее знаки, значимые для оболочки, не может вызвать непредусмотренного исполнения.

**Замечание**

> Круглые скобки в `/PARAMETERS=(...)` истолкует оболочка, вызывающая `batch`. Заключайте весь квалификатор в одиночные кавычки, как показано выше, либо обходитесь без скобок: `/PARAMETERS=full,/srv/data,verbose`.

Каждому заданию даются ещё три переменные окружения:

| Переменная | Содержимое |
|---|---|
| `BATCH_ENTRY` | Номер записи задания |
| `BATCH_NAME` | Имя задания |
| `BATCH_QUEUE` | Имя очереди |
| `BATCH_RESTART` | `TRUE`, если задание выполняется повторно после перезапуска контроллера |
| `BATCH_RESTARTS` | Сколько раз задание перезапускалось |

## 4.3 Вывод задания

У пакетного задания нет терминала. Его стандартный ввод присоединён к пустому устройству, а стандартный вывод и стандартный поток ошибок направлены в *журнал задания*.

Если квалификатор `/LOG_FILE` не указан, журнал создаётся в подкаталоге `log` корневого каталога подсистемы и называется по имени задания и номеру его записи:

```
/var/spool/batch/log/nightly_17.log
```

Судьба журнала зависит от того, как завершилось задание:

| Исход | Запись очереди | Журнал |
|---|---|---|
| Код возврата нулевой | Снимается | Удаляется, если не указан `/KEEP` |
| Код возврата не нулевой | Сохраняется | Сохраняется |
| Завершено по сигналу | Сохраняется | Сохраняется |
| Любой исход при `/RETAIN=ALWAYS` | Сохраняется | Сохраняется |

Именно это правило и определяет поведение команды `SHOW` по умолчанию: в очереди остаётся в точности то, что ещё выполняется либо не удалось.

Чтобы журнал успешного задания сохранился для просмотра, указывайте `/KEEP`:

```
$ batch submit /opt/jobs/nightly.sh /KEEP
```

## 4.4 Управление временем запуска

### 4.4.1 Отсрочка

Квалификатор `/AFTER` не даёт заданию начаться раньше назначенного времени. Задание переводится в состояние `Holding` и освобождается само, когда время наступает:

```
$ batch submit /opt/jobs/nightly.sh '/AFTER=+1-02:00'
$ batch submit /opt/jobs/report.sh  /AFTER=29-JUL-2026:03:00
$ batch submit /opt/jobs/cleanup.sh /AFTER=TOMORROW
```

Допустимые формы времени полностью описаны в Приложении B.

### 4.4.2 Удержание и освобождение

Квалификатор `/HOLD` помещает задание в очередь, но не даёт рассматривать его для исполнения, пока оно не будет освобождено явно:

```
$ batch submit /opt/jobs/migrate.sh /NAME=migrate /HOLD
Job migrate (queue SYS$BATCH, entry 19) queued

$ batch release 19
Entry 19: operation completed
```

Освобождение снимает и связанное с заданием время `/AFTER`, поэтому освобождённое задание становится готовым немедленно.

### 4.4.3 Приоритет

Квалификатор `/PRIORITY` определяет порядок выбора среди ожидающих заданий. Контроллер выбирает ожидающее задание с наибольшим приоритетом, а среди равных по приоритету — с наименьшим номером записи, так что задания одного приоритета обслуживаются в порядке подачи.

```
$ batch submit /opt/jobs/urgent.sh /PRIORITY=200
```

**Замечание**

> `/PRIORITY` — приоритет задания *в очереди*. На приоритет планирования порождённого для задания процесса он не влияет и, когда задание уже выполняется, не значит ничего. Привязку к процессорам задаёт `/CPU`; значение nice, ограничения ресурсов и управляющая группа наследуются от контроллера и для отдельного задания не задаются.

### 4.4.4 Изменение задания в очереди

Что решила `SUBMIT`, решено не навсегда. Команда `SET ENTRY` изменяет атрибуты задания, стоящего в очереди, на месте: запись сохраняет номер и место в порядке, а затрагиваются лишь атрибуты, названные в командной строке.

```
$ batch set entry 19 /PRIORITY=200
%BATCH-S-ENTMODIFIED, entry 19 modified: priority=200

$ batch set entry 19 /AFTER=TOMORROW /NOHOLD
%BATCH-S-ENTMODIFIED, entry 19 modified: after= 9-SEP-2026 00:00:00, nohold
```

У ожидающего задания можно изменить всё. У исполняющегося — лишь то, что касается его дальнейшей судьбы: `/RETAIN`, `/KEEP`, `/RESTART`; у сохранённого — ничего. Правила полностью приведены в словаре команд.

### 4.4.5 Повторное исполнение после перезапуска

Квалификатор `/RESTART` предписывает выполнить задание снова, если контроллер был остановлен, пока задание выполнялось. Такое задание возвращается в очередь при следующем пуске контроллера; без этого квалификатора оно остаётся в состоянии `Retained`, помеченным как осиротевшее, и повторно не выполняется.

Процесс, оставшийся от прежнего экземпляра контроллера, не разыскивается и не снимается: он живёт в своём сеансе и вполне мог уже отработать. Квалификатор обещает, что задание будет выполнено, а не что оно будет выполнено однажды, — поэтому перезапускаемое задание следует писать так, чтобы двойное исполнение не вредило.

## 4.5 Просмотр очереди

Команда `SHOW` показывает очередь. Без квалификаторов она показывает задания, которые выполняются, снимаются либо сохранены после неудачи:

```
$ batch show

 Entry  Job name            State      Submitted            Elapsed   Status
 -----  ------------------- ---------- -------------------- --------- ----------
    17  nightly             Executing  29-JUL-2026 10:01:05 00:04:12  -
    18  reindex             Retained   29-JUL-2026 10:01:05 00:00:07  exit=7

Total entries: 2
```

Столбец `Status` показывает `exit=`*n* для задания, вернувшего ненулевой код, `signal=`*n* для завершённого по сигналу, и дефис в остальных случаях.

Ожидающие задания (`Pending`, `Holding`) показываются тоже. Квалификатор `/ALL` добавляет записи успешно завершённых заданий, оставленные `/RETAIN=ALWAYS`:

```
$ batch show /all
```

Квалификатор `/FULL` вместо строки на задание выводит его полное описание:

```
$ batch show /full

Entry 17, job nightly, queue SYS$BATCH
    State ............ Executing, pid 4412
    Owner ............ smith (uid 1000, gid 1000)
    Priority ......... 100
    Processors ....... 0-3
    Script ........... /opt/jobs/nightly.sh
    Working directory  /home/smith
    Log file ......... /var/spool/batch/log/nightly_17.log
    Submitted ........ 29-JUL-2026 10:01:05
    Started .......... 29-JUL-2026 10:01:05
    Parameters ....... P1 = "full"
                       P2 = "/srv/data"
```

По умолчанию показываются только собственные задания. Квалификатор `/EVERYONE` показывает задания всех пользователей.

## 4.6 Снятие задания

Команда `DELETE` снимает запись очереди. Её действие зависит от состояния задания:

| Состояние | Действие |
|---|---|
| `Pending`, `Holding`, `Retained` | Запись снимается, журнал удаляется, если не был указан `/KEEP` |
| `Executing` | Заданию посылается сигнал снятия; запись обрабатывается по завершении процесса |

```
$ batch delete 17
Entry 17: operation completed
```

Сигнал посылается *группе процессов* задания, а не одному процессу. Задание — лидер своей группы, поэтому вместе с ним снимается всё, что оно породило.

Снимать можно только собственные задания; привилегированный пользователь — любые.

# Глава 5. Словарь команд

Здесь каждая команда описана подробно. Команды таковы:

```
BATCH SUBMIT
BATCH SHOW
BATCH DELETE
BATCH RELEASE
BATCH SET ENTRY
BATCHD
```

Командную строку разбирают средства CLI из подпроекта UTILITY$ROUTINES. Квалификатор вводится косой чертой либо дефисом и сокращается до любого однозначного начала: `/PARAM=` и `/PARAMETERS=` равнозначны, а неоднозначное сокращение отвергается с перечислением подходящих. `/RESTART` и `/RETAIN` совпадают первыми двумя буквами, поэтому сокращение любого из них требует трёх. Значение присоединяется к квалификатору знаком равенства.

Параметр команды и её квалификаторы могут следовать в любом порядке.

Значения проверяются по объявленному для квалификатора типу: `/PRIORITY` и `/ENTRY` принимают только число, `/RETAIN` — только одно из своих ключевых слов, `/CPU` — только правильный список процессоров.

Образ `batch` возвращает оболочке 0, если операция выполнена, и 1, если нет.

# BATCH SUBMIT

Помещает сценарий оболочки в очередь для исполнения пакетным заданием.

## Формат

```
batch submit  спецификация-сценария
```

## Параметр

### спецификация-сценария

Задаёт сценарий оболочки, который надлежит выполнить. Спецификация разворачивается в абсолютный путь при вводе команды. Файл должен существовать и быть доступен для чтения и тому, кто вводит команду, и контроллеру заданий.

Поскольку и спецификация файла, и квалификатор начинаются с косой черты, команда различает их по имени: довод, начинающийся с косой черты, считается квалификатором лишь тогда, когда текст до первого знака равенства или двоеточия называет известный квалификатор. Поэтому `/tmp/job.sh` распознаётся как спецификация файла, а `/NAME=x` — как квалификатор.

## Описание

Команда SUBMIT заводит запись в пакетной очереди и сообщает назначенный ей номер. Исполнения задания она не ожидает.

Следующие свойства фиксируются при подаче и позднее не изменяются:

- абсолютный путь сценария;
- рабочий каталог, бывший текущим при вводе команды;
- личность подавшего, от имени которой задание и будет работать.

Если очередь достигла потолка либо сценарий недоступен, команда отказывает, и запись не заводится.

## Квалификаторы

### /AFTER=*время*

Задаёт время, раньше которого задание начинаться не должно. Задание переводится в состояние `Holding` до наступления этого времени, после чего контроллер освобождает его без чьего-либо участия. Допустимые формы описаны в Приложении B.

### /CPU=*список-процессоров*

Привязывает задание к названным процессорам в той записи, какую применяет `taskset(1)`: перечисление номеров и диапазонов через запятую, `0-3,5,8-11`. Без этого квалификатора задание наследует маску привязки контроллера, которую тот сообщает в своём журнале при запуске.

Список проверяется при вводе команды, поэтому ошибка в нём сообщается на терминал, а не превращается в упавшее задание.

### /HOLD

Помещает задание в состояние `Holding`. Оно не рассматривается для исполнения, пока не будет освобождено командой `RELEASE`. Применяйте, когда время запуска ещё не известно.

### /KEEP

Сохраняет журнал задания после успешного завершения. По умолчанию журнал успешного задания удаляется вместе с его записью очереди.

### /LOG_FILE=*спецификация-файла*

Задаёт файл, в который направляются стандартный вывод и стандартный поток ошибок задания. Спецификация разворачивается в абсолютный путь при вводе команды; сам файл существовать не обязан, а вот его каталог обязан. По умолчанию журнал создаётся в подкаталоге `log` корневого каталога подсистемы и называется *имя-задания*`_`*номер*`.log`.

### /NAME=*имя-задания*

Задаёт имя задания, до 63 знаков. По умолчанию имя задания есть имя файла сценария без окончания `.sh`. Имя задания появляется в выводе и входит в имя журнала, принятое по умолчанию.

### /PARAMETERS=(*значение*[,...])

Задаёт до восьми значений, передаваемых заданию. Каждое — до 255 знаков. Значения даются сценарию и переменными окружения `P1` … `P8`, и позиционными доводами `$1` … `$8`. Если значений больше восьми, лишние отбрасываются с предупреждением; задание всё равно ставится в очередь.

### /PRIORITY=*n*

Задаёт приоритет задания в очереди, от 1 до 255. По умолчанию 100. Контроллер выбирает ожидающее задание с наибольшим приоритетом; задания одного приоритета выбираются в порядке номеров записей. На приоритет порождённого процесса этот квалификатор не влияет.

### /QUEUE=*имя-очереди*

Задаёт очередь, в которую помещается задание. По умолчанию — та, с которой запущен контроллер, обычно `SYS$BATCH`.

### /RESTART

Предписывает выполнить задание снова, если контроллер был остановлен, пока задание выполнялось. Такое задание возвращается в очередь при следующем пуске контроллера; без квалификатора оно остаётся в состоянии `Retained`, помеченным как осиротевшее, и повторно не выполняется.

Процесс, оставшийся от прежнего экземпляра контроллера, не разыскивается и не снимается: он живёт в своём сеансе и вполне мог уже отработать. Квалификатор обещает, что задание будет выполнено, а не что оно будет выполнено однажды, поэтому перезапускаемое задание следует писать так, чтобы двойное исполнение не вредило.

Заданию сообщают о перезапуске переменные окружения `BATCH_RESTART`, принимающая значения `TRUE` и `FALSE`, и `BATCH_RESTARTS`, считающая перезапуски. Это соответствует символу `$RESTART` в OpenVMS.

### /USER=*имя*

Выполняет задание от имени названного пользователя, а не подавшего. Просить об этом вправе только привилегированный пользователь: личность устанавливает контроллер по тому, что о посылающем сообщает ядро, а не по тому, что говорит запрос.

Всякое применение квалификатора и всякий отказ записываются в журнал контроллера с указанием, кто просил и для кого.

Задание начинается в домашнем каталоге названного пользователя, а не там, откуда подано, и его журнал уходит в каталог журналов подсистемы.

### /RETAIN={ALWAYS | ERROR | NEVER}

Задаёт условие, при котором запись сохраняется после завершения задания. Ключевое слово можно сокращать до трёх знаков.

| Слово | Действие |
|---|---|
| `ALWAYS` | Запись сохраняется всегда |
| `ERROR` | Запись сохраняется только при неудаче задания. Принято по умолчанию. |
| `NEVER` | Запись не сохраняется, даже если задание не удалось |

## Примеры

1.

```
$ batch submit /opt/jobs/nightly.sh
Job nightly (queue SYS$BATCH, entry 17) queued
```

Команда помещает сценарий `nightly.sh` в очередь. Задание берёт имя от сценария, попадает в очередь по умолчанию и готово к исполнению немедленно. Номер записи 17 опознаёт его в дальнейших командах.

2.

```
$ batch submit /opt/jobs/backup.sh /NAME=weekly '/PARAMETERS=(full,/srv/data)' \
        '/AFTER=+1-02:00' /PRIORITY=150 /KEEP
Job weekly (queue SYS$BATCH, entry 18) queued
```

Помещает в очередь задание с именем `weekly`, которое не может начаться раньше, чем через 26 часов. Сценарию передаются два параметра. Приоритет выше принятого по умолчанию, поэтому, когда время придёт, задание будет выбрано прежде заданий обычного приоритета. Журнал сохранится, даже если задание завершится успешно.

3.

```
$ batch submit /opt/jobs/migrate.sh /HOLD
Job migrate (queue SYS$BATCH, entry 19) queued
$ batch show
...
    19  migrate              Holding    29-JUL-2026 11:20:03 00:00:00  -
```

Задание помещено в очередь, но удержано от исполнения. Оно ожидает, поэтому показывается и без `/ALL`.

# BATCH SHOW

Показывает содержимое пакетной очереди.

## Формат

```
batch show
```

## Параметры

Отсутствуют.

## Описание

Команда SHOW показывает записи очереди. Какие именно — определяют квалификаторы /ALL и /EVERYONE; сколько показывать о каждой — квалификатор /FULL.

Без /ALL показываются задания в состояниях Executing, Aborting, Pending и Holding и сохранённые (Retained) после неудачи: те, что выполняются, те, что ждут исполнения, и те, что требуют внимания. Запись успешно завершённого задания очередь покидает. Если задание подано с /RETAIN=ALWAYS, запись остаётся и показывается только с /ALL.

В кратком выводе столбец Elapsed показывает время от начала задания до его завершения либо до настоящего момента, если задание ещё выполняется.

## Квалификаторы

### /ALL

Добавляет к выводу записи успешно завершённых заданий, оставленные `/RETAIN=ALWAYS`.

### /EVERYONE

Показывает задания всех пользователей. По умолчанию показываются только задания того, кто вводит команду.

### /FULL

Выводит полное описание каждого задания: владельца, приоритет, привязку к процессорам, спецификацию сценария, рабочий каталог, спецификацию журнала, отметки времени, код завершения и параметры.

### /QUEUE=*имя-очереди*

Ограничивает вывод заданиями названной очереди.

## Примеры

1.

```
$ batch show

 Entry  Job name             State      Submitted            Elapsed   Status
 -----  -------------------- ---------- -------------------- --------- ----------
    17  nightly              Executing  29-JUL-2026 10:01:05 00:04:12  -
    18  reindex              Retained   29-JUL-2026 10:01:05 00:00:07  exit=7

Total entries: 2
```

Одно задание выполняется; другое завершилось с кодом 7 и сохранено в очереди вместе со своим журналом.

2.

```
$ batch show /all /everyone /queue=SYS$BATCH
```

Показывает все записи очереди `SYS$BATCH`, в любом состоянии, всех пользователей.

# BATCH DELETE

Снимает запись очереди, завершая задание, если оно выполняется.

## Формат

```
batch delete  список-записей  [/CONFIRM]
```

## Параметр

### номер-записи

Задаёт номер записи задания, показанный командами `SUBMIT` и `SHOW`. Номер можно указать и квалификатором `/ENTRY=`, который имеет преимущество.

## Квалификатор

### /CONFIRM

Спрашивает, прежде чем действовать. Всё, кроме слова на Y, — отказ: сделанное командой DELETE не отменить, а список умножает цену ошибки. Перенаправленный ввод считается подтверждённым — вопрос,
ушедший в файл, ждал бы нажатия, которого некому сделать, — и `/NOCONFIRM` говорит то же явно.

## Описание

Если задание выполняется, его группе процессов посылается сигнал снятия, и задание переходит в состояние Aborting; запись обрабатывается по завершении процесса, а журнал сохраняется. В иных случаях запись снимается сразу, а её журнал удаляется, если при подаче не был указан /KEEP.

Снимать можно только собственные задания. Привилегированный пользователь — любые.

## Пример

```
$ batch delete 17
Entry 17: operation completed
```

# BATCH RELEASE

Освобождает задание из состояния `Holding`.

## Формат

```
batch release  список-записей
```

## Параметр

### номер-записи

`Задаёт номер записи задания.`

## Описание

Задание переводится в состояние `Pending` и становится готовым к исполнению. Связанное с ним время `/AFTER` снимается, поэтому освобождённое задание не удерживается второй раз.

Команда отказывает, если задание не находится в состоянии `Holding`.

## Пример

```
$ batch release 19
Entry 19: operation completed
```

# BATCH SET ENTRY

Изменяет атрибуты задания, стоящего в очереди, на месте.

## Формат

```
batch set entry  список-записей  [/PRIORITY=n] [/AFTER=время | /NOAFTER] [/HOLD | /NOHOLD]
                                 [/NAME=имя] [/RETAIN=ключ] [/KEEP | /NOKEEP]
                                 [/RESTART | /NORESTART] [/CPU=список]
```

## Параметр

### список-записей

`Записи, которые надо изменить; записывается как у DELETE: номера и диапазоны через запятую.`

## Описание

Запись сохраняет номер и место в порядке; ничто не снимается и не подаётся заново. Затрагиваются лишь атрибуты, названные квалификаторами, и подтверждение перечисляет их, чтобы можно было сверить с набранным. Команда, не называющая ни одного, отвергается.

Что можно изменить, зависит от состояния задания:

| Состояние | Можно изменить |
|---|---|
| `Pending`, `Holding` | всё |
| `Executing`, `Aborting` | только `/RETAIN`, `/KEEP`, `/RESTART` |
| `Retained` | ничего |

Запрос, называющий атрибут, который состояние запрещает, отвергается целиком, чтобы запись никогда не оказалась изменённой наполовину.

`/HOLD` и `/AFTER` действуют вместе так же, как при подаче. `/HOLD` удерживает ожидающее задание; `/NOHOLD` отпускает удержанное и снимает его время, как `RELEASE`. `/AFTER` с будущим временем удерживает ожидающее задание до этого времени. Удержанное задание со временем отпускается, когда время наступает или когда `/NOAFTER` его снимает; удержанное одним `/HOLD` ждёт `RELEASE` или `/NOHOLD`. Поэтому `/HOLD` вместе с `/NOAFTER` означает: удержано, без времени. Когда даны оба, сначала применяется удержание, затем время, так что `/NOHOLD /AFTER=t` отпускает задание и удерживает его до t.

## Квалификаторы

**/PRIORITY=n** — приоритет в очереди, от 1 до 255.

**/AFTER=время**, **/NOAFTER** — время, раньше которого задание не запускается, в форматах приложения B; `/NOAFTER` его снимает.

**/HOLD**, **/NOHOLD** — удержать задание или отпустить удержанное.

**/NAME=имя** — имя задания, до 63 символов. Меняет то, что показывают выводы, и ничего больше: файл журнала был назван при подаче и имени не меняет.

**/RETAIN=ключ** — `ALWAYS`, `ERROR` или `NEVER`, как у SUBMIT.

**/KEEP**, **/NOKEEP** — хранить ли журнал успешно завершившегося задания.

**/RESTART**, **/NORESTART** — выполнять ли задание снова после перезапуска контроллера.

**/CPU=список** — процессоры, к которым привязано задание, в записи SUBMIT.

## Примеры

```
$ batch set entry 17 /PRIORITY=200
%BATCH-S-ENTMODIFIED, entry 17 modified: priority=200

$ batch set entry 17 /HOLD /NOAFTER
%BATCH-S-ENTMODIFIED, entry 17 modified: noafter, hold

$ batch set entry 17 /NAME=x
%BATCH-E-SUBREJECT, the controller rejected the request: NOTMODIFIABLE (08010362)
```

Последнее — задание, которое уже исполняется: его имя описывает состоявшийся запуск.

# BATCH PURGE

Снимает записи, оставшиеся от завершившихся заданий.

```
BATCH PURGE [/QUEUE=очередь] [/CONFIRM]
```

Очередь, сохраняющая свои неудачи, нуждается в уборке: запись, оставленная после неудавшегося задания, стоит, пока в неё не посмотрят, — а посмотрели, и записи остались.

Берётся только завершившееся задание. Выполняющееся не трогается: PURGE прибирает то, что кончилось, а не останавливает то, что нет. Свои записи, если просит не привилегированный пользователь, — тот вправе распорядиться любыми.

```
$ batch purge
%BATCH-S-PURGED, 3 entries removed from the queue
```

| Квалификатор | Действие |
|---|---|
| `/QUEUE=`*`очередь`* | Ограничивает уборку названной очередью |
| `/CONFIRM` | Спрашивает, прежде чем действовать |

# BATCH SYNC

Ожидает конца задания и сообщает, чем оно кончилось.

```
BATCH SYNC номер-записи [/TIMEOUT=секунд]
```

Предназначена для сценария, которому есть что делать после задания и неоткуда узнать, когда оно готово. Ответ несёт код возврата, так что читать вывод вызывающему не нужно: ноль — задание удалось, не ноль — не удалось, время вышло либо такой записи нет.

```
batch submit nightly.sh /NAME=nightly
batch sync 17 /TIMEOUT=3600 || echo "ночное задание не задалось"
```

Запись, покинувшая очередь, считается успешной — очередь покидают именно так. Без `/TIMEOUT` ожидание не ограничено.

# BATCH QUEUE

Приостанавливает очередь или пускает её дальше. Просить об этом вправе только привилегированный пользователь: приостановка удерживает работу всех, кто очередью пользуется.

```
BATCH QUEUE имя-очереди {/PAUSE | /RESUME}
```

Приостановленная очередь ничего не запускает. Записи её не трогаются, а выполняющиеся задания доводятся до конца: приостановка — то, что делают с очередью, задания которой должны подождать, а записи остаться нетронутыми.

# BATCH ANALYZE

Разбирает журнал очереди и сообщает, что с ним не так.

```
BATCH ANALYZE [спецификация-журнала] [/DUMP]
```

Файл читается напрямую, а не через контроллер, поэтому команда служит там, где контроллер не запускается, — то есть тогда, когда она нужнее всего, — и равно служит на копии журнала, снятой с другой машины. Ничего не пишется.

`/DUMP` показывает и сами записи: номер, имя, размер, состояние, владельца, приоритет, очередь, время постановки и исполняемый сценарий, чем кончилось завершившееся задание и чего ждёт удержанное.

# BATCHD

Запускает контроллер заданий.

## Формат

```
batchd  [/ROOT=каталог] [/JOB_LIMIT=n] [/QUEUE=имя] [/FOREGROUND] [/VERBOSE]
        [/LOG_RETAIN=дней]
```

## Описание

Контроллер владеет очередью. Он должен работать, иначе ни одна команда `batch` не выполнится.

При запуске контроллер создаёт, если нужно, свой корневой каталог и подкаталог журналов, восстанавливает очередь из журнала и создаёт сокет, на котором принимает запросы. Задания, которые выполнялись при остановке предыдущего экземпляра, помечаются осиротевшими и переводятся в состояние `Retained` — их процессы не являются потомками нового экземпляра, и код завершения получить уже нельзя. Задание, поданное с `/RESTART`, вместо этого возвращается в очередь и выполняется снова.

Если не указан `/FOREGROUND`, контроллер отцепляется от терминала и направляет диагностику в файл `batchd.log` в корневом каталоге подсистемы. Этот журнал перематывается по достижении мегабайта; журнал отдельного задания — нет, и удаляется он только вместе с записью, которой принадлежит.

## Квалификаторы

### /FOREGROUND

Работать на переднем плане, не подменяя стандартные потоки. Для проверок и для работы под супервизором служб.

### /JOB_LIMIT=*n*

Задаёт число одновременно выполняющихся заданий. По умолчанию 4, независимо от числа процессоров. Значения вне диапазона от 1 до 512 заменяются принятым по умолчанию. Значение, не являющееся числом, отвергается при разборе.

### /LOG_RETAIN=*дней*

Задаёт, сколько хранится оставшийся в каталоге журналов журнал задания. Контроллер обходит этот каталог при запуске и далее раз в час, удаляя файлы старше указанного срока, не принадлежащие ни одной записи в очереди. По умолчанию 7 дней; ноль отключает уборку.

Журнал, названный квалификатором `/LOG_FILE` вне каталога журналов подсистемы, не трогается никогда.

### /QUEUE=*имя-очереди*

Задаёт имя очереди по умолчанию. По умолчанию `SYS$BATCH`.

### /ROOT=*каталог*

Задаёт корневой каталог подсистемы, замещая значение, установленное при сборке программы.

### /VERBOSE

Выдаёт и сообщения информационной серьёзности.

### /QUEUES

Показывает очереди, а не задания: каждую с её пределом, с тем, что в ней выполняется, ждёт и удержано, и приостановлена ли она.

```
 Queue                Limit  Exec  Pend  Hold  State
 -------------------- ----- ----- ----- ----- --------
 SYS$BATCH               16     0     0     0  open
 slow                     1     1     1     0  paused
```

### /MANAGER

Показывает состояние контроллера заданий, а не содержимое очереди: образ, которым он работает, его процесс и сколько времени он поднят, обслуживаемый корневой каталог, предел заданий и сколько из него занято, счёт по состояниям, следующий номер записи и пул процессоров.

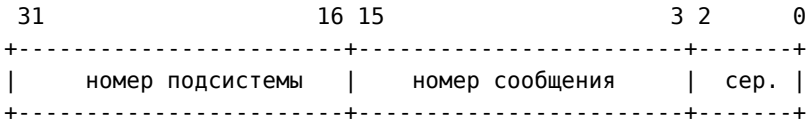
## Примеры

```
# batchd /ROOT=/var/spool/batch /JOB_LIMIT=8
# batchd -r=/var/spool/batch -j=8
```

# Глава 6. Сообщения

## 6.1 Раскладка значения условия

Каждая рутина подсистемы возвращает *значение условия* — длинное слово, опознающее и исход, и его причину. Раскладка принята из OpenVMS и объявлена подпроектом UTILITY$ROUTINES.




| Поле | Биты | Содержимое |
|---|---|---|
| `SEV` | 0–2 | Серьёзность |
| `MSG_NO` | 3–15 | Номер сообщения в пределах подсистемы |
| `FAC_NO` | 16–31 | Номер подсистемы |
| `CUST_DEF` | 27 | Взведён, если номер подсистемы назначен не изготовителем системы |

Коды серьёзности:

| Значение | Серьёзность | Признак успеха |
|---|---|---|
| 0 | Предупреждение | снят |
| 1 | Успех | взведён |
| 2 | Ошибка | снят |
| 3 | Информация | взведён |
| 4 | Неустранимая ошибка | снят |

Младший бит значения условия и есть признак успеха, поэтому исход любой рутины проверяется независимо от того, какое именно условие она вернула.

## 6.2 Вид сообщения

Сообщения имеют такой вид:

```
29-07-2026 10:58:15.171  1206 %BATCH-E-NOSCRIPT, script /nonexistent.sh: not found, realpath()->-1, errno: 2 (No such file or directory)
```

Поля таковы: отметка времени, идентификатор процесса, имя подсистемы, односимвольная мнемоника серьёзности, мнемоника условия и текст сообщения. При действующем ключе `/VERBOSE` вслед идёт вторая строка, называющая модуль и рутину, обнаружившие условие.

Значение условия, выработанное контроллером, передаётся утилите в ответе. Само сообщение утилита выдаёт как значение, а полный его текст остаётся там, где он был выработан, — в журнале контроллера. Поэтому сообщение о ненайденной записи очереди печатает `batch`, хотя условие выработал `batchd`.

Отказ рутины библиотеки времени исполнения сообщается там, где он произошёл, с именем рутины, её итогом и числовым значением `errno` — чтобы не потерять единственное, что позволяет отличить одну причину отказа от другой.

## 6.3 Справочник сообщений

Ниже перечислены все сообщения подсистемы в порядке возрастания номеров. Текст приведён так, как он записан в таблице сообщений: директива с восклицательным знаком — `!AZ` для строки, `!UL` или `!SL` для числа, `!8XL` для шестнадцатеричного значения — отмечает место, куда подставляется значение. `!AZ`, приставленная к слову, как в `entr!AZ` или `second!AZ`, даёт его окончание, а после числа — добавляет к нему пояснение. Сами сообщения программа выдаёт по-английски, поэтому здесь они не переведены — иначе читатель не нашёл бы в справочнике то, что увидел на экране.

### Общие

**NORMAL**, успех

```
normal successful completion
```

*Действие:* Не требуется.

**INFO**, информация

```
the controller runs on processors !AZ, and a job inherits that
```

*Действие:* Не требуется. Сообщается один раз, при запуске контроллера.

**BUGCHECK**, неустранимая ошибка

```
internal inconsistency: !AZ
```

*Действие:* Дефект подсистемы. Сообщите о нём, приведя текст сообщения и журнал контроллера.

### Рутины общего назначения

**BUFOVF**, ошибка

```
result of !SL bytes does not fit in !UL
```

*Действие:* Укоротите путь или значение, которое не поместилось. Если пути принадлежат самой подсистеме, со

**INVTIME**, ошибка

```
unparsable time specification /AFTER=!AZ
```

*Действие:* Исправьте спецификацию времени. Допустимые формы перечислены в Приложении B.

**MKDIRERR**, ошибка

```
cannot create !AZ, mkdir()->!SL, errno: !SL (!AZ)
```

*Действие:* Убедитесь, что родительский каталог существует и что контроллер вправе в нём писать.

**IOERR**, ошибка

```
error transferring data, !AZ()->!SL, errno: !SL (!AZ)
```

*Действие:* Обмен с другой стороной прервался. Повторите; если повторяется, посмотрите журнал контроллера.

**ENDOFDATA**, ошибка

```
data ended earlier than expected, !AZ()->!SL after !UL of !UL bytes
```

*Действие:* Другая сторона закрыла соединение раньше срока. Повторите. Снятый по сигналу клиент даёт это со

**INVSTATE**, предупреждение

```
unknown job state
```

*Действие:* Дефект подсистемы. Сообщите о нём.

**CWDERR**, ошибка

```
getcwd()->NULL, errno: !SL (!AZ)
```

*Действие:* Текущий каталог удалён или недоступен для чтения. Перейдите в существующий каталог.

**SMALLBUF**, ошибка

```
supplied buffer is manifestly too small
```

*Действие:* Дефект подсистемы. Сообщите о нём.

## Очередь и задания

**JOBQUED**, успех

```
job !UL (!AZ) queued on !AZ, state !AZ
```

*Действие:* Не требуется. Запомните номер записи — он нужен командам DELETE и RELEASE.

**JOBSTART**, успех

```
job !UL (!AZ) started, pid=!SL, log !AZ
```

*Действие:* Не требуется.

**JOBDONE**, успех

```
job !UL (!AZ) completed successfully in !SQ s
```

*Действие:* Не требуется. Запись покинула очередь.

**JOBFAIL**, ошибка

```
job !UL (!AZ) terminated with status !SL!AZ, log !AZ
```

*Действие:* Посмотрите журнал задания, названный в сообщении.

**JOBABORT**, ошибка

```
job !UL (!AZ) terminated by signal !SL, log !AZ
```

*Действие:* Заданию послан сигнал. Если это сделано не намеренно, посмотрите журнал задания и журнал системы

**JOBORPHAN**, предупреждение

```
job !UL was orphaned by a restart of the job controller
```

*Действие:* Задание выполнялось, когда контроллер остановился, и его исход неизвестен. Подайте задание снов

**NOSLOT**, ошибка

```
the job table is full, capacity !SL entries, while !AZ
```

*Действие:* Очередь достигла потолка. Удалите ненужные записи или поднимите BATCH_K_MAXJOB и пересоберите.

**NOSUCHQUE**, ошибка

```
there is no queue named !AZ
```

*Действие:* Очередь не из тех, о которых контроллеру сообщили. SHOW /QUEUES их перечисляет; квалификатор /QU

**NOTCONF**, информационное

```
not confirmed, nothing was done
```

*Действие:* На вопрос /CONFIRM отвечено не словом на Y. Ничего не сделано; повторите команду, если она была н

**PURGED**, успех

```
!UL entr!AZ removed from the queue
```

*Действие:* Не требуется. PURGE снял записи завершившихся заданий.

**QUEORPHAN**, предупреждение

```
queue !AZ was not configured; restored paused for its entries
```

*Действие:* В журнале нашлись записи очереди, о которой контроллеру не сообщили. Очередь восстановлена, и во

**QUEPAUSED**, успех

```
queue !AZ is paused
```

*Действие:* Не требуется. Очередь ничего не запускает, пока не будет возобновлена.

**SCRIPTCHG**, ошибка

```
entry !UL: script !AZ was replaced since it was submitted
```

*Действие:* Сценарий не тот файл, что был подан. Он закрепляется при подаче по устройству, узлу, размеру и вр

**SYNCFAIL**, ошибка

```
job !UL ended with !AZ
```

*Действие:* Задание, которого ждала команда SYNC, не удалось. Что сказал сценарий, лежит в его журнале.

**SYNCTMO**, ошибка

```
job !UL has not ended within !UL second!AZ
```

*Действие:* Время, заданное /TIMEOUT, вышло. Задание по-прежнему в очереди; SHOW скажет, в каком оно состоян

**NOSUCHENT**, ошибка

```
no queue entry !UL
```

*Действие:* Проверьте номер записи командой SHOW /ALL. Запись успешно завершённого задания очередь покидает

**NOPRIV**, ошибка

```
entry !UL belongs to uid=!SL, access denied
```

*Действие:* Запись принадлежит другому пользователю. Распоряжаться ею вправе её владелец либо привилегирова

**NOTHELD**, ошибка

```
entry !UL is not in the holding state
```

*Действие:* RELEASE применима к заданию в состоянии Holding. Состояние каждой записи показывает SHOW /ALL.

**ENTDELETED**, успех

```
entry !UL deleted from the queue
```

*Действие:* Не требуется.

**ENTRELEASED**, успех

```
entry !UL released from the holding state
```

*Действие:* Не требуется.

**ENTMODIFIED**, успех

```
entry !UL modified: !AZ
```

*Действие:* Не требуется. Список называет изменённые атрибуты; сверьте его с набранным.

**NOTMODIFIABLE**, ошибка

```
entry !UL is !AZ, the attribute cannot be altered now
```

*Действие:* Состояние задания запрещает изменение: у исполняющегося допустимы лишь /RETAIN, /KEEP и /RESTAR
ничего. Если изменение необходимо, снимите запись и подайте задание заново.

**NOTHINGSET**, ошибка

```
no attribute to set was given
```

*Действие:* Команде SET ENTRY не дано ни одного квалификатора. Назовите атрибут, который надо изменить.

**ABORTREQ**, успех

```
SIGTERM sent to job !UL (pid=!SL)
```

*Действие:* Не требуется. Запись будет обработана, когда задание завершится.

**JOBREQUEUED**, успех

```
job !UL (!AZ) requeued for restart, attempt !UL
```

*Действие:* Не требуется. Задание выполняется повторно; переменная BATCH_RESTART сообщает ему об этом.

## Файлы и журнал очереди

**OPENIN**, ошибка

```
error opening !AZ for input, open()->!SL, errno: !SL (!AZ)
```

*Действие:* Убедитесь, что файл существует и доступен контроллеру для чтения.

**OPENOUT**, ошибка

```
error opening !AZ for output, open()->!SL, errno: !SL (!AZ)
```

*Действие:* Убедитесь, что каталог существует, на файловой системе есть место, и контроллер вправе там писат

**WRITEERR**, ошибка

```
error writing !AZ, errno: !SL (!AZ)
```

*Действие:* Скорее всего, файловая система заполнена. Освободите место и повторите.

**RENAMEERR**, ошибка

```
cannot rename !AZ to !AZ, rename()->!SL, errno: !SL (!AZ)
```

*Действие:* Журнал очереди не удалось поставить на место. Проверьте место и права на корневом каталоге подси

**NOJOURNAL**, информация

```
journal !AZ does not exist, the queue is empty
```

*Действие:* Не требуется. Контроллер начинает с пустой очереди, что при первом запуске и ожидается.

**JRNCORRUPT**, ошибка

```
journal !AZ is corrupt: the header is missing
```

*Действие:* Журнал непригоден, очередь начинается пустой. Отложите файл в сторону и сообщите о случае.

**JRNVERSION**, ошибка

```
journal !AZ is incompatible: magic=!8XL format=!UL
```

*Действие:* Журнал написан другой редакцией программы. Удалите его; очередь начнётся пустой.

**JRNTRUNC**, предупреждение

```
journal !AZ ends at record !UL of !UL
```

*Действие:* Журнал оборван, скорее всего заполненной файловой системой. Прочитанные записи сохранены, остал

**JRNLOADED**, информация

```
records loaded from the journal: !UL
```

*Действие:* Не требуется.

**NOSCRIPT**, ошибка

```
script !AZ: !AZ, !AZ()->!SL, errno: !SL (!AZ)
```

*Действие:* Проверьте путь, существование файла, что это обычный файл, и что он доступен для чтения и вам, и

**LOGSPEC**, ошибка

```
the job log specification cannot be constructed
```

*Действие:* Спецификация журнала не поместилась в отведённое место. Укоротите имя задания или корневой катал

**NODIR**, ошибка

```
directory !AZ is not accessible, realpath()->NULL, errno: !SL (!AZ)
```

*Действие:* Проверьте каталог, названный в /LOG_FILE.

**LOGSWEPT**, информация

```
!SL job log!AZ removed from !AZ, older than !SL day!AZ
```

*Действие:* Не требуется. Срок задаётся ключом /LOG_RETAIN, ноль отключает уборку.

## Обмен между образами

**NOCONNECT**, ошибка

```
job controller is not available, connect(!AZ)->!SL, errno: !SL (!AZ)
```

*Действие:* Контроллер не запущен либо работает с другим корневым каталогом. Запустите службу и проверьте BA

**SOCKERR**, неустранимая ошибка

```
socket(AF_UNIX)->!SL, errno: !SL (!AZ)
```

*Действие:* Система отказала в сокете. Посмотрите предел на число открытых файлов.

**BINDERR**, неустранимая ошибка

```
bind(!AZ)->!SL, errno: !SL (!AZ)
```

*Действие:* Оставшийся файл сокета либо права доступа. Проверьте корневой каталог подсистемы.

**LISTENERR**, неустранимая ошибка

```
listen(!AZ)->!SL, errno: !SL (!AZ)
```

*Действие:* Сообщите о случае, приведя текст сообщения.

**ACCEPTERR**, ошибка

```
accept()->!SL, errno: !SL (!AZ)
```

*Действие:* Обычно случайность. Если повторяется, посмотрите предел на число открытых файлов.

**PEERCRED**, ошибка

```
getsockopt(SO_PEERCRED)->!SL, errno: !SL (!AZ)
```

*Действие:* Личность клиента установить не удалось, запрос отвергнут. Сообщите о случае.

**BADMAGIC**, ошибка

```
invalid frame signature: !8XL
```

*Действие:* К сокету подключилось что-то, кроме утилиты, либо утилита и контроллер разных редакций.

**BADLENGTH**, ошибка

```
invalid !AZ payload length: !UL
```

*Действие:* Утилита и контроллер разных редакций. Установите их из одной сборки.

**BADFUNC**, ошибка

```
unknown function code !UL
```

*Действие:* Утилита и контроллер разных редакций.

**NOREPLY**, ошибка

```
no reply to the !AZ request
```

*Действие:* Контроллер остановился, пока запрос был в пути. Повторите.

**SENDERR**, ошибка

```
error transmitting !AZ
```

*Действие:* Соединение прервалось. Повторите.

**RECVERR**, ошибка

```
error receiving !AZ
```

*Действие:* Соединение прервалось. Повторите.

**SOCKPATH**, неустранимая ошибка

```
socket specification !AZ exceeds !UL bytes
```

*Действие:* Корневой каталог слишком длинен для пути сокета. Выберите короче.

**CONNCLOSED**, предупреждение

```
client uid=!SL closed the connection before sending a header
```

*Действие:* Не требуется. Такое сообщение даёт снятый клиент.

## Разбор командной строки

**IVQUAL**, ошибка

```
unrecognized or ambiguous qualifier !AZ
```

*Действие:* Исправьте квалификатор. Допустимые для каждой команды перечислены в Главе 5.

**MISSVAL**, ошибка

```
qualifier /!AZ requires a value
```

*Действие:* Укажите значение: квалификатор его требует, присоединяя знаком равенства.

**IVQVAL**, ошибка

```
invalid value /RETAIN=!AZ
```

*Действие:* Исправьте значение. Что принимает квалификатор, сказано в описании команды.

**IVPRIO**, ошибка

```
the value of /PRIORITY is outside 1..!SL
```

*Действие:* Укажите приоритет от 1 до 255.

**MAXPARM**, предупреждение

```
more than !SL parameters, the excess were discarded
```

*Действие:* Задание поставлено с первыми восемью параметрами. Передайте меньше или передайте их файлом.

**NOPARAM**, ошибка

```
no script specification was given
```

*Действие:* Укажите спецификацию исполняемого скрипта.

**EXTRAPARM**, ошибка

```
extraneous command parameter: !AZ
```

*Действие:* Команда принимает не более одного параметра. Квалификатор с опечаткой принимается за параметр,

**IVCPU**, ошибка

```
unparsable processor list /CPU=!AZ
```

*Действие:* Исправьте список процессоров. Запись такая же, как у taskset(1): 0-3,5,8-11.

**IVVERB**, ошибка

```
unrecognized command
```

*Действие:* Пользуйтесь командами SUBMIT, SHOW, DELETE или RELEASE.

**NOENTRY**, ошибка

```
no queue entry number was given
```

*Действие:* Укажите номер записи — позиционно либо ключом /ENTRY.

**SUBREJECT**, ошибка

```
the controller rejected the request: !AZ (!8XL)
```

*Действие:* Контроллер отверг запрос; полная причина изложена в его журнале.

## Порождение процессов и работа контроллера

**FORKERR**, ошибка

```
job !UL, fork()->!SL, errno: !SL (!AZ)
```

*Действие:* Системе не удалось породить процесс. Посмотрите предел на число процессов и доступную память.

**DAEMONERR**, неустранимая ошибка

```
daemon()->!SL, errno: !SL (!AZ)
```

*Действие:* Сообщите о случае, приведя текст сообщения.

**PIPEERR**, неустранимая ошибка

```
pipe()->!SL, errno: !SL (!AZ)
```

*Действие:* Посмотрите предел на число открытых файлов.

**POLLERR**, ошибка

```
poll()->!SL, errno: !SL (!AZ)
```

*Действие:* Сообщите о случае, приведя текст сообщения.

**AFFERR**, ошибка

```
job !UL: sched_setaffinity()->!SL, errno: !SL (!AZ)
```

*Действие:* Названные процессоры недоступны контроллеру. Маску, под которой он работает, он сообщает в своё

**UNKPROC**, предупреждение

```
an unknown process terminated, pid=!SL
```

*Действие:* Обычно не требуется. Завершился процесс, не принадлежащий ни одной записи.

**CTLSTARTED**, успех

```
job controller started, root=!AZ, queue=!AZ, job limit=!SL, socket=!AZ
```

*Действие:* Не требуется.

**CTLSTOPPING**, успех

```
job controller is stopping
```

*Действие:* Не требуется.

**IDENT**, успех

```
!AZ !AZ (!AZ), build !AZ
```

*Действие:* Не требуется.

# Глава 7. Устранение неполадок

В каждом разделе указан признак, куда смотреть и что делать. Два места стоит знать заранее — журнал контроллера и журнал отдельного задания.

```
# journalctl -u batchd              # когда контроллер работает службой
# cat /var/spool/batch/batchd.log   # когда он запущен вручную и отцепился
$ batch show /full                  # состояние каждой записи в очереди
```

# 7.1 Контроллер недоступен

```
$ batch show
%BATCH-E-NOCONNECT, job controller is not available, connect(/var/spool/batch/batch.sock)-
>-1,
errno: 2 (No such file or directory)
```

Сообщение называет сокет, который искала утилита. Причины стоит проверять по порядку.

**Контроллер не запущен.**

```
# systemctl status batchd
# systemctl start batchd
```

**Он работает с другим корневым каталогом.** Утилита смотрит в `BATCH_ROOT`, а если переменная не установлена — в значение, заданное при сборке; контроллер пользуется тем, что указано ему ключом `/ROOT`. Сравните:

```
$ echo "${BATCH_ROOT:-<не установлена, берётся значение из сборки>}"
# systemctl show batchd -p ExecStart | tr ' ' '\n' | grep ROOT
```

**Сокет есть, но открыть его нельзя** — `errno: 13 (Permission denied)`. Сокет создаётся с режимом 0666, а доступ решается по личности отправителя, поэтому дело в *каталоге*: он не пропускает. Проверьте, что каждая составляющая пути к корневому каталогу проходима для пользователя.

## 7.2 Контроллер не запускается

Читайте его первые сообщения: он называет причину и останавливается.

```
%BATCH-F-BINDERR, bind(/var/spool/batch/batch.sock)->-1, errno: 98 (Address already in use)
```

На этом корневом каталоге уже работает другой контроллер. Один контроллер владеет одним корневым каталогом; двое стали бы спорить за журнал.

```
%BATCH-E-MKDIRERR, cannot create /var/spool/batch, mkdir()->-1, errno: 13 (Permission denied)
```

Учётная запись, от имени которой он работает, не вправе создать корневой каталог. Либо создайте его заранее с нужным владельцем, либо запускайте контроллер от имени того, кто вправе.

```
%BATCH-E-JRNVERSION, queue journal is incompatible with this revision
```

Журнал написан другой редакцией программы. Очередь начинается пустой; отложите файл в сторону или удалите его. Такое ожидаемо после обновления, изменившего дескриптор задания.

## 7.3 Задание не начинается

Причину показывает SHOW: столбец State ожидающего задания говорит, чего оно ждёт.

```
$ batch show
 Entry  Job name             State      Submitted            Elapsed   Status
 -----  -------------------- ---------- -------------------- --------- ----------
     7  nightly              Pending    29-JUL-2026 10:01:05 00:00:00  -
```

| Состояние | Что означает | Что делать |
|---|---|---|
| Pending | Ждёт свободного места для исполнения | Другие задания выполняются. SHOW /ALL /EVERYONE покажет, сколько их; поднимите /JOB_LIMIT, если предел тесен. |
| Holding | Удержано ключом /HOLD либо /AFTER | Освободите командой RELEASE *номер*. Время /AFTER показывает SHOW /FULL. |
| В списке нет вовсе | Запись покинула очередь | Успешно завершённое задание очередь покидает. Подавайте с /RETAIN=ALWAYS, если запись должна остаться. |

Задание с /AFTER в прошлом начинается сразу; с /AFTER в будущем ждёт, причём показываемое время — местное. Задание, которое ждёт дольше назначенного, стоит сверить с TZ: контроллер и спрашивающий могут читать разные часы, и объясняется это обыкновенно именно этим.

## 7.4 Задание завершилось неудачей

Запись остаётся в очереди, исход показан в столбце `Status`, а всё, что задание написало, — в его журнале.

```
$ batch show
     7  nightly              Retained   29-JUL-2026 10:01:05 00:00:12  exit=127

$ batch show /full            # среди прочего - спецификация журнала
$ cat /var/spool/batch/log/nightly_7.log
```

| Исход | Обычная причина |
|---|---|
| `exit=126` | Задание вообще не удалось начать: интерпретатор, рабочий каталог, журнал либо список процессоров. См. ниже. |
| `exit=127` | Интерпретатор не нашёл того, что требует сценарий. Обычно дело в `PATH`: заданию даются `BATCH_*` и `P1` … `P8` и ничего более, ваше окружение оно не наследует. |
| `exit=`*n* | Сценарий сам вернул этот код. Его собственное дело. |
| `signal=15` | Задание снято командой `DELETE` либо остановлено системой. |
| `signal=9` | Убито наотрез, чаще всего распорядителем памяти. |
| `exit=-1` | Контроллер остановился, пока задание работало, и исход неизвестен; сообщение — `%BATCH-W-JOBORPHAN`. Подавайте с `/RESTART`, чтобы такое задание выполнялось снова. |

Разобрать стоит `exit=126`, поскольку задание не написало ничего: отказ произошёл прежде, чем сценарий начался. По убыванию вероятности — рабочий каталог подачи больше не существует, журнал не создать, список `/CPU` называет процессоры, недоступные контроллеру, либо не удалось принять личность подавшего.

Снимайте запись, когда она больше не нужна; журнал уйдёт вместе с ней, если не был указан `/KEEP`:

```
$ batch delete 7
```

## 7.5 После перезапуска очередь пуста

Ожидаемо в трёх случаях, и ни в одном из них это не дефект.

- Задание, завершившееся **успешно**, покинуло очередь при завершении; так и задумано. Запись сохраняет `/RETAIN=ALWAYS`.
- Журнал **отвергнут**: `%BATCH-E-JRNVERSION` после обновления либо `%BATCH-E-JRNCORRUPT`. Контроллер говорит об этом при запуске.
- Контроллер работает с **другим корневым каталогом**, не с тем, где лежит журнал.

Задание, которое *выполнялось*, не теряется, но помечается `exit=-1` и на этом останавливается — если оно не несло `/RESTART`, в каковом случае оно ставится в очередь снова и узнаёт об этом из `BATCH_RESTART`.

## 7.6 Каталог журналов растёт

Журналы заданий удаляются вместе с их записями очереди. Двух видов это не касается: журналов заданий, поданных с `/KEEP`, и журналов заданий, чьи записи не пережили грубой остановки. Контроллер обходит их через `/LOG_RETAIN` дней, по умолчанию семь, — при запуске и далее раз в час.

```
%BATCH-I-LOGSWEPT, 5 job logs removed from /var/spool/batch/log, older than 7 days
```

Укоротите срок, если каталог всё равно растёт; ничего не удаляется, пока журналом владеет запись в очереди, сколь бы стар он ни был. Журнал, названный `/LOG_FILE` вне каталога журналов, не трогается никогда — это дело того, кто его назвал.

## 7.7 Команда отвергнута

```
$ batch submit /opt/jobs/nightly.sh /RE=ALWAYS
%CLI_RTNS-F: Ambiguous input 'RE' (matched to : 'RESTART', 'RETAIN')
```

Сообщения под именем `%CLI_RTNS-` исходят от средств разбора командной строки и касаются формы команды; сообщения под `%BATCH-` исходят от подсистемы и касаются её смысла. Сокращение должно быть однозначным, а `/RESTART` и `/RETAIN` совпадают первыми двумя буквами.

```
$ batch submit job.sh /NOSUCH=1
%BATCH-E-EXTRAPARM, extraneous command parameter: /NOSUCH=1
```

Довод, начинающийся с косой черты и не называющий квалификатора, считается спецификацией файла — именно это и позволяет `/tmp/job.sh` быть путём. Поэтому квалификатор с опечаткой выглядит как лишний параметр; проверьте написание.

## 7.8 Контроллер медленно отвечает

Очередь целиком записывается в журнал при каждом изменении, и запись сбрасывается на носитель. На большой очереди это и есть основная трата. Если контроллер отстаёт при череде подач, очередь, скорее всего, велика: `SHOW /ALL /EVERYONE` покажет, насколько, а снятие ненужных записей укоротит каждую последующую запись журнала.

Один клиент задержать контроллер не может: соединения обслуживаются без блокировки, а тот, кто замолчал посреди запроса, снимается через тридцать секунд.

## 7.9 Что собрать для сообщения об ошибке

Приведите редакцию программы, сообщение целиком и то, что делалось:

```
$ batch                                    # идентификация в первой строке
# journalctl -u batchd --since '1 hour ago' > batchd.txt
$ batch show /all /everyone /full > queue.txt
```

Сообщение серьёзности `F` с мнемоникой `BUGCHECK` есть дефект подсистемы по определению, и сообщать о нём стоит всегда.

# Приложение A. Файлы

Корневой каталог подсистемы задаётся при сборке программы и замещается ключом `/ROOT` контроллера, а для утилиты — переменной окружения `BATCH_ROOT`. По умолчанию это `/var/spool/batch`.

| Файл | Содержимое |
|---|---|
| *корень*`/batch.sock` | Сокет, на котором контроллер принимает запросы |
| *корень*`/journal.dat` | Журнал очереди |
| *корень*`/batchd.log` | Журнал самого контроллера, когда он отцеплен от терминала |
| *корень*`/log/` | Журналы заданий, для которых не указан `/LOG_FILE` |

Журнал очереди пишется во временный файл и переименовывается поверх прежней копии, поэтому в недописанном виде он не наблюдается никогда. Журнал содержит двоичный образ каждой записи очереди; если раскладка записи изменилась между редакциями программы, журнал прежней редакции отвергается, и очередь начинается пустой.

Сокет создаётся с режимом 0666. Доступ к заданиям решается не правами файла, а личностью отправителя, которую сообщает ядро и которая проверяется при каждом запросе.

# Переменные окружения

| Переменная | Кто читает | Назначение |
|---|---|---|
| `BATCH_ROOT` | `batch` | Корневой каталог подсистемы |
| `BATCH_ENTRY` | Задание | Номер записи задания |
| `BATCH_NAME` | Задание | Имя задания |
| `BATCH_QUEUE` | Задание | Имя очереди |
| `BATCH_RESTART` | Задание | `TRUE`, если задание выполняется снова после перезапуска контроллера |
| `BATCH_RESTARTS` | Задание | Сколько раз задание перезапускалось |
| `P1` … `P8` | Задание | Параметры задания |

# Приложение B. Формы спецификации времени

Квалификатор `/AFTER` принимает три формы записи времени.

## B.1 Разностное время

Разностное время начинается со знака плюс и означает промежуток, отсчитываемый от момента ввода команды:

```
+[дддд-]чч[:мм[:сс]]
```

Опущенные поля считаются нулевыми.

| Запись | Промежуток |
|---|---|
| `+3-` | Трое суток |
| `+3` | Три часа |
| `+:30` | Тридцать минут |
| `+::45` | Сорок пять секунд |
| `+1-12:00` | Сутки и двенадцать часов |
| `+0-00:05:30` | Пять минут тридцать секунд |

**Замечание**

Поле суток отличается от поля часов дефисом, который за ним следует, и ничем иным: `+7` — это семь часов, а `+7-` — семь суток. Поскольку ошибка здесь не обнаруживается, пока срок не истечёт, подача называет время, с которого задание начнётся.

## B.2 Абсолютное время

Абсолютное время означает определённый момент и читается по часам того, кто его вводит:

```
дд-МММ-гггг[:чч:мм:сс]
```

Месяц — трёхбуквенное сокращение в любом регистре. Опущенные поля времени считаются нулевыми, поэтому одна дата означает полночь, с которой эти сутки начинаются.

| Запись | Смысл |
|---|---|
| `29-JUL-2026` | Полночь, с которой начинается 29 июля 2026 года |
| `29-JUL-2026:03:00:00` | 03:00 29 июля 2026 года |

## B.3 Ключевые слова

| Слово | Смысл |
|---|---|
| `NOW` | Настоящий момент |
| `TODAY` | Полночь, с которой начинаются текущие сутки |
| `TOMORROW` | Полночь, с которой начинаются следующие сутки |

Поскольку `TODAY` означает уже миновавший момент, задание, поданное с `/AFTER=TODAY`, готово к исполнению немедленно.